

Rapport de projet — Motus Microservices

Master 2 MIAGE SITN — Apprentissage 2025-2026
Module : Applications Web orientées Services — M. Menceur
Soutenance / remise : 4 juillet 2026
Dépôt GitHub : github.com/Aaronath/motus-microservice-miage

1. Binôme

Étudiant 1	Étudiant 2
Halioua Nathan	Trabelsi Yacob

Répartition du travail

Domaine	Nathan Halioua	Yacob Trabelsi
Conception de l'architecture microservices	✓	✓
Services Spring Boot (backend)	✓	
Interface React (frontend)	✓	✓
Docker Compose et scripts de build	✓	
Recette fonctionnelle et conformité au sujet	✓	✓
Rédaction du rapport et de la documentation	✓	✓

2. Compilation et exécution

Voir le fichier `README.md` à la racine du dépôt GitHub.

Résumé :

```
./scripts/build-all.sh
docker compose up --build -d
# Frontend : http://localhost:3000
# API Gateway : http://localhost:8080
```

Compte administrateur : code `MIAGE-ADMIN-2026` à l'inscription.

3. Documentation technique

3.1 Schéma d'architecture

Architecture en 5 composants backend + 1 client React :

- **api-gateway** : point d'entrée, routage, CORS
- **player-service** : gestion des joueurs et JWT
- **dictionary-service** : dictionnaire français, validation des mots
- **game-service** : parties, propositions, algorithme Motus
- **stats-service** : historique, scores, classement

Chaque service métier possède sa **base PostgreSQL** (principe *database per service*).

Schéma détaillé : `docs/ARCHITECTURE.md` .

3.2 Choix techniques

Aspect	Choix	Justification
Framework	Spring Boot 3.2	Stack cours, écosystème mature
Persistence	Spring Data JPA	Exigence énoncé
SGBD	PostgreSQL 16	Relationnel, Docker/K8s
API	REST JSON	Interopérabilité, simplicité démo
Auth	JWT (jjwt)	Stateless, adapté microservices
Gateway	Spring Cloud Gateway	Routage centralisé
Client	React + Vite	Démo visuelle convaincante
Conteneurs	Docker Compose	Déploiement reproductible
Orchestration	Minikube / K8s	Manifests fournis (k8s/) ; démo validée via Docker Compose

3.3 Diagramme de classes métier

Entités principales :

- **Joueur** (id, pseudo, email, rôle, dateInscription)
- **Partie** (joueur, motMystère, essaisMax, statut, dates)
- **Proposition** (mot, numéroEssai, feedback lettres)
- **RésultatPartie** (gagnée, nombreEssais, score)
- **MotDictionnaire** (mot, longueur)

Relations : un Joueur joue plusieurs Parties ; une Partie contient plusieurs Propositions ; une Partie produit un RésultatPartie.

Diagramme Mermaid : docs/CLASS_DIAGRAM.md .

3.4 Règles du jeu implémentées

1. Première lettre toujours affichée (FIRST)
2. Proposition refusée si longueur incorrecte, mauvaise 1ère lettre, ou mot absent du dictionnaire
3. Scoring : bien placées puis mal placées (gestion des doublons)
4. Victoire si mot exact ; défaite si essais épuisés
5. Longueur des mots : 5 à 9 lettres (lexique Lexique383)

3.5 APIs REST

Documentation complète : docs/API.md .

Conformité au cahier des charges : docs/CONFORMITE_SUJET.md .

4. Bilan du projet

Ce que nous avons apprécié

Nous avons particulièrement apprécié de **mettre en pratique les notions du cours** — découpage en microservices, persistance JPA, API Gateway — sur un cas concret et ludique. L'implémentation de **l'algorithme Motus** avec un retour visuel immédiat (grille colorée, animations) a donné une dimension tangible au projet. Enfin, la **stack Docker Compose** nous a permis de disposer d'une démonstration fiable et reproductible, aussi bien en développement qu'en vue de la soutenance.

Ce que nous avons appris

Ce projet nous a permis d'approfondir le **découpage d'un domaine en bounded contexts** (joueurs, parties, dictionnaire, statistiques), chacun avec sa propre base de données. Nous avons mis en œuvre la **communication inter-services** via RestClient (game → dictionary, game → stats) et une **authentification JWT stateless** partagée entre la gateway et les services métier. Nous avons consolidé nos compétences en **Spring Data JPA**, en **conteneurisation multi-services** (quatre PostgreSQL, cinq services Java, un frontend Nginx) et en **routage centralisé** avec Spring Cloud Gateway.

Points plus difficiles

Les principales difficultés ont concerné **l'orchestration de quatre bases PostgreSQL** en parallèle et la cohérence des variables d'environnement entre conteneurs. La **gestion homogène des erreurs HTTP** à travers la gateway et les services a également demandé de l'attention. L'**import du lexique** (~42 000 mots issus de Lexique383) au démarrage du dictionary-service a nécessité un traitement spécifique (compression, indexation). Enfin, les **manifests Kubernetes** fournis restent simplifiés ; le déploiement complet sur Minikube n'a pas été finalisé, la validation s'appuyant sur Docker Compose.

Réussites

Nous sommes satisfaits d'avoir livré une **application jouable de bout en bout** : inscription, création de partie, propositions, scoring, historique et classement. Le **panneau d'administration** dépasse le minimum exigé (recherche de parties, gestion des joueurs et du dictionnaire). Le dictionnaire s'appuie sur **Lexique383** avec des groupes thématiques, et des **tests unitaires** couvrent l'algorithme de scoring. Une **documentation structurée** (architecture, API, conformité) accompagne le code source.

Pistes d'amélioration

À terme, un **service discovery** (Eureka ou Consul) éviterait les URLs codées en dur, et un **bus de messages** (Kafka) pourrait découpler l'enregistrement des statistiques. Un **pipeline CI/CD** avec tests d'intégration automatisés (déjà esquissés dans le module `integration-tests`) renforcerait la qualité. Enfin, un **déploiement Kubernetes complet** avec les quatre bases dédiées permettrait de valider l'orchestration sur Minikube.